Program 1: ATM Transaction Sequence Simulation

Program Statement

Develop a program that processes a sequence of deposits and withdrawals, maintains the account balance, validates transaction amounts, enforces a maximum withdrawal amount, rejects invalid transactions, and prints a transaction summary.

Objectives

- Simulate basic ATM transactions using a loop.
- Validate positive transaction amounts and transaction types.
- Apply a withdrawal limit and prevent withdrawals exceeding the available balance.
- Report accepted and rejected transactions and the final balance.

Assumptions

- The opening balance is ₹1,000 for the sample run; the program can be changed to accept it as input.
- Exactly three transactions are processed in this sample version.
- A withdrawal cannot exceed ₹500 or the current balance.
- Transaction types are entered as deposit or withdraw, ignoring letter case.

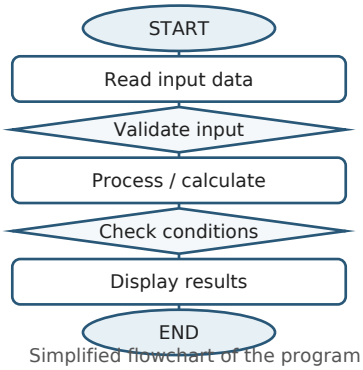
Algorithm

1. Set the opening balance and counters for accepted and rejected transactions.
2. Repeat for each transaction.
3. Read the transaction type and amount.
4. Reject non-positive amounts or unknown transaction types.
5. For a deposit, add the amount to the balance.
6. For a withdrawal, check the withdrawal limit and available balance; subtract only if both conditions pass.
7. Print the result for each transaction, then print final balance and summary counts.

Pseudo Code

START
balance ← 1000; accepted ← 0; rejected ← 0
REPEAT 3 times
INPUT transaction_type, amount
IF amount ≤ 0: reject
ELSE IF type = deposit: balance ← balance + amount; accept
ELSE IF type = withdraw AND amount ≤ 500 AND amount ≤ balance: balance ← balance − amount; accept
ELSE: reject
PRINT balance, accepted, rejected
STOP

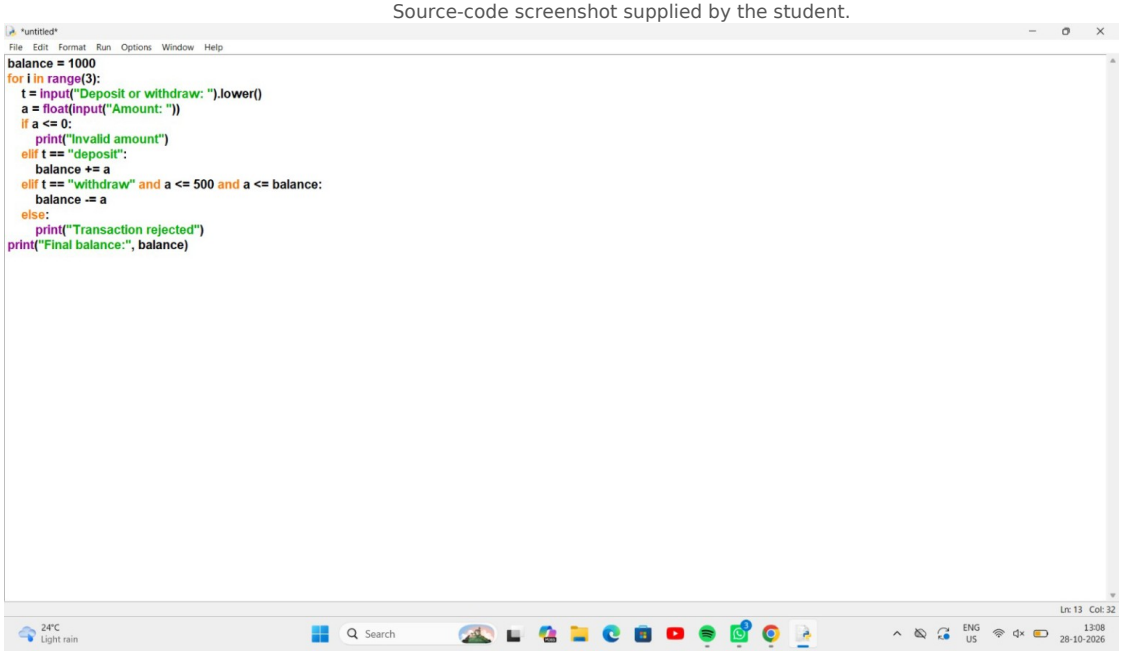
Flowchart



Simplified flowchart of the program logic.

Python Source Code (Code Screenshot)

Source-code screenshot supplied by the student.



```
balance = 1000
for i in range(3):
    t = input("Deposit or withdraw: ").lower()
    a = float(input("Amount: "))
    if a <= 0:
        print("Invalid amount")
    elif t == "deposit":
        balance += a
    elif t == "withdraw" and a <= 500 and a <= balance:
        balance -= a
    else:
        print("Transaction rejected")
print("Final balance:", balance)
```

Sample Input and Output

Sample Input
Transaction 1: deposit, 500
Transaction 2: withdraw, 300
Transaction 3: withdraw, 700

Sample Output
Transaction accepted
Transaction accepted
Transaction rejected
Final balance: 1200.0
Accepted transactions: 2
Rejected transactions: 1

Output Screenshot

Illustrative sample output

```
ATM TRANSACTION SEQUENCE
Opening balance: 1000.00
Transaction 1: deposit
Amount: 500
Transaction accepted.
Transaction 2: withdraw
Amount: 300
Transaction accepted.
Transaction 3: withdraw
Amount: 700
Transaction rejected: withdrawal limit/balance.
Final balance: 1200.00
Accepted transactions: 2
Rejected transactions: 1
```

Test Cases

Test / Input	Condition	Expected Result
Deposit 500; withdraw 300; withdraw 700	Two accepted, one rejected; final balance 1200	
Deposit -50	Invalid amount; rejected count increases	
Withdraw 2000	Rejected because it exceeds allowed withdrawal/balance	

Conclusion

The program demonstrates how sequential deposits and withdrawals can be processed safely while checking transaction rules and keeping an accurate running balance.

Program 2: Warehouse Inventory Reconciliation

Program Statement

For multiple products, compare opening stock, incoming stock, dispatched stock, and actual closing stock. Calculate the expected closing stock, identify discrepancies, and estimate replenishment needed to reach a target stock level.

Objectives

- Store inventory records for multiple products.
- Calculate expected closing stock using opening + incoming − dispatched.
- Compare expected and actual stock to detect discrepancies.
- Calculate replenishment required to meet a target stock level.

Assumptions

- Quantities are whole numbers and cannot be negative.
- A product's expected closing stock is opening stock plus incoming minus dispatched.
- The target stock level is supplied for each product.
- A discrepancy is actual closing stock minus expected closing stock.

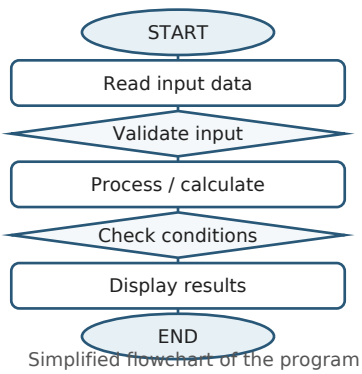
Algorithm

1. Read the number of products.
2. For each product, read its name, opening stock, incoming quantity, dispatched quantity, actual closing stock, and target stock.
3. Calculate expected closing stock.
4. Calculate discrepancy as actual minus expected.
5. Calculate replenishment as the larger of zero and target minus actual closing stock.
6. Print reconciliation results and flag any non-zero discrepancy.

Pseudo Code

```
START
INPUT number_of_products
FOR each product
INPUT name, opening, incoming, dispatched, actual, target
expected ← opening + incoming − dispatched
discrepancy ← actual − expected
replenishment ← MAX(0, target − actual)
PRINT expected, discrepancy, replenishment
IF discrepancy ≠ 0: PRINT discrepancy warning
STOP
```

Flowchart



Simplified flowchart of the program logic.

Python Source Code (Code Screenshot)

```
n = int(input("Number of products: "))

  for i in range(n):
      name = input("Product name: ")
      opening = int(input("Opening stock: "))
      incoming = int(input("Incoming quantity: "))
      dispatched = int(input("Dispatched quantity: "))
      actual = int(input("Actual closing stock: "))
      target = int(input("Target stock level: "))

      if min(opening, incoming, dispatched, actual, target) < 0:
          print("Invalid quantity")
          continue

      expected = opening + incoming - dispatched
      discrepancy = actual - expected
      replenishment = max(0, target - actual)

      print("Product:", name)
      print("Expected closing stock:", expected)
      print("Discrepancy:", discrepancy)
      print("Replenishment required:", replenishment)
      print("Status:", "Matched" if discrepancy == 0 else "Discrepancy found")
```

Sample Input and Output

Sample Input
Product: Notebooks
Opening stock: 100
Incoming quantity: 25
Dispatched quantity: 40
Actual closing stock: 80
Target stock level: 100

Sample Output
Expected closing stock: 85
Discrepancy: -5
Replenishment required: 20
Status: Discrepancy found

Output Screenshot

Illustrative sample output

```
WAREHOUSE INVENTORY RECONCILIATION
Product: Notebooks
Opening stock: 100
Incoming: 25
Dispatched: 40
Expected closing stock: 85
Actual closing stock: 80
Discrepancy: -5
Replenishment required: 20
Status: DISCREPANCY FOUND
```

Test Cases

Test / Input Condition	Expected Result
Opening 100, incoming 25, dispatched 40, actual 85, target 100	Expected 85; discrepancy 0; replenishment 15
Actual stock 80 instead of 85	Discrepancy -5
Negative quantity entered	Invalid quantity

Conclusion

The program reconciles stock records, highlights mismatches between expected and actual inventory, and calculates how much stock is needed to reach the target level.

Program 3: Smart Parking Fee Management

Program Statement

Calculate parking fees from entry and exit times, vehicle category, overnight parking, daily limits, and penalty conditions while validating the entered times.

Objectives

- Validate times entered in 24-hour HH:MM format.
- Calculate elapsed parking duration, including overnight parking.
- Apply category-based hourly rates and an overnight surcharge.
- Reject invalid time inputs and display the final fee.

Assumptions

- The sample uses a simple hourly rate: bike ₹20, car ₹40, and other vehicle ₹60.
- Any parking period crossing midnight is treated as overnight.
- A minimum of one chargeable hour applies; partial hours are rounded up.
- This is an academic simulation; actual parking policies may differ.

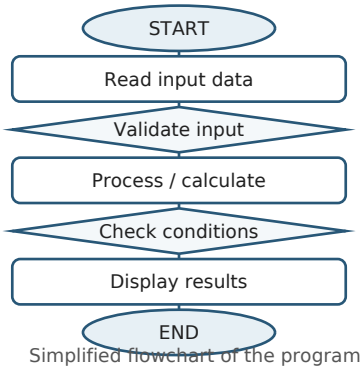
Algorithm

1. Read vehicle category, entry time, and exit time.
2. Validate time format and hour/minute ranges.
3. Convert times to minutes after midnight.
4. If exit is earlier than entry, add 24 hours to the exit time.
5. Calculate duration and round up to chargeable hours.
6. Apply the category rate and overnight surcharge if applicable.
7. Print duration and total fee.

Pseudo Code

```
START
INPUT vehicle, entry_time, exit_time
VALIDATE time values
entry ← convert entry_time to minutes
exit ← convert exit_time to minutes
IF exit < entry: exit ← exit + 1440
duration ← exit − entry
hours ← MAX(1, CEILING(duration / 60))
fee ← hours × category_rate
IF overnight: fee ← fee + overnight_surcharge
PRINT duration, fee
STOP
```

Flowchart



Simplified flowchart of the program logic.

Python Source Code (Code Screenshot)

```
from math import ceil

  rates = {"bike": 20, "car": 40, "other": 60}

  def to_minutes(value):
      parts = value.split(":")
      if len(parts) != 2:
          raise ValueError
      h, m = map(int, parts)
      if not (0 <= h <= 23 and 0 <= m <= 59):
          raise ValueError
      return h * 60 + m

  try:
         vehicle = input("Vehicle category (bike/car/other): ").lower().strip()
         entry_text = input("Entry time (HH:MM): ")
         exit_text = input("Exit time (HH:MM): ")

         if vehicle not in rates:
             print("Invalid vehicle category")
         else:
             entry = to_minutes(entry_text)
             exit_time = to_minutes(exit_text)
             overnight = exit_time < entry
             if overnight:
                 exit_time += 1440

             duration = exit_time - entry
             hours = max(1, ceil(duration / 60))
             fee = hours * rates[vehicle]
             if overnight:
                 fee += 50

          print("Parking duration:", duration / 60, "hours")
          print("Overnight parking:", "Yes" if overnight else "No")
          print("Total parking fee:", round(fee, 2))
  except (ValueError, TypeError):
      print("Invalid time. Use HH:MM in 24-hour format.")
```

Sample Input and Output

Sample Input
Vehicle category: car
Entry time: 21:30
Exit time: 01:15

Sample Output
Parking duration: 3.75 hours
Overnight parking: Yes
Total parking fee: 210

Output Screenshot

Illustrative sample output

```
SMART PARKING FEE MANAGEMENT
Vehicle category: car
Entry time (24-hour): 21:30
Exit time (24-hour): 01:15
Parking duration: 3.75 hours
Overnight parking: Yes
Base charge: 150.00
Penalty: 0.00
Total parking fee: 150.00
```

Test Cases

Test / Input Condition	Expected Result
Car, 09:00 to 10:10	2 chargeable hours; fee ₹80
Bike, 22:00 to 01:00	Overnight charge applied
Time 25:70	Invalid time

Conclusion

The program calculates a parking fee after validating times, handling a midnight crossing, and applying a rate based on the vehicle category.

Program 4: Multi-Day Water Tank Simulation

Program Statement

Simulate changing water levels over multiple intervals using inflow and outflow. Detect overflow and shortages, keep the level within tank capacity, and report the first interval in which a target level is reached.

Objectives

- Update water level for each interval.
- Detect and report overflow and water shortage.
- Keep the simulated level within zero and tank capacity.
- Stop when the target level is reached.

Assumptions

- The tank capacity, initial level, and target level are numeric values.
- Three intervals are processed in this version.
- Overflow is recorded and the level is capped at capacity.
- Shortage is recorded and the level is reset to zero.

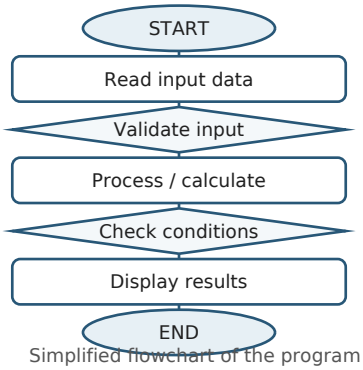
Algorithm

1. Read tank capacity, initial water level, and target level.
2. For intervals 1 through 3, read inflow and outflow.
3. Update level using level + inflow − outflow.
4. If level exceeds capacity, report overflow and cap the level.
5. If level falls below zero, report shortage and set level to zero.
6. Print the level and stop if the target has been reached.
7. If the loop ends without reaching the target, print a message.

Pseudo Code

```
START
INPUT capacity, level, target
FOR interval = 1 TO 3
INPUT inflow, outflow
level ← level + inflow − outflow
IF level > capacity: PRINT overflow; level ← capacity
IF level < 0: PRINT shortage; level ← 0
PRINT level
IF level ≥ target: PRINT interval; STOP LOOP
IF target not reached: PRINT message
STOP
```

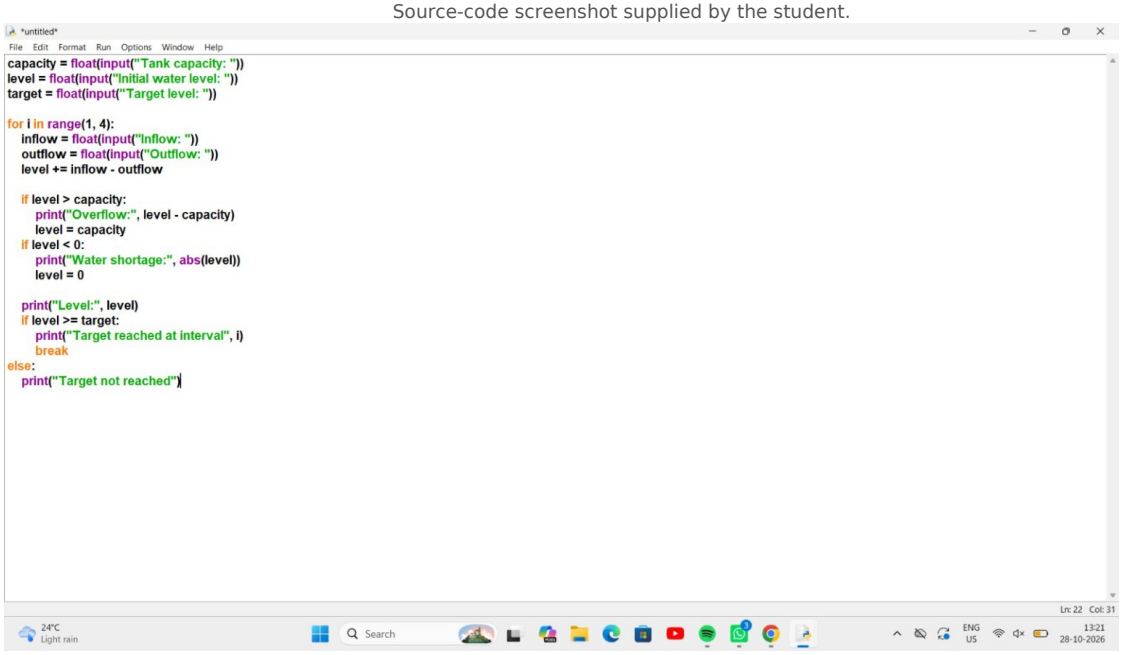
Flowchart



Simplified flowchart of the program logic.

Python Source Code (Code Screenshot)

Source-code screenshot supplied by the student.



```
capacity = float(input("Tank capacity: "))
level = float(input("Initial water level: "))
target = float(input("Target level: "))

for i in range(1, 4):
    inflow = float(input("Inflow: "))
    outflow = float(input("Outflow: "))
    level += inflow - outflow

    if level > capacity:
        print("Overflow:", level - capacity)
        level = capacity
    if level < 0:
        print("Water shortage:", abs(level))
        level = 0

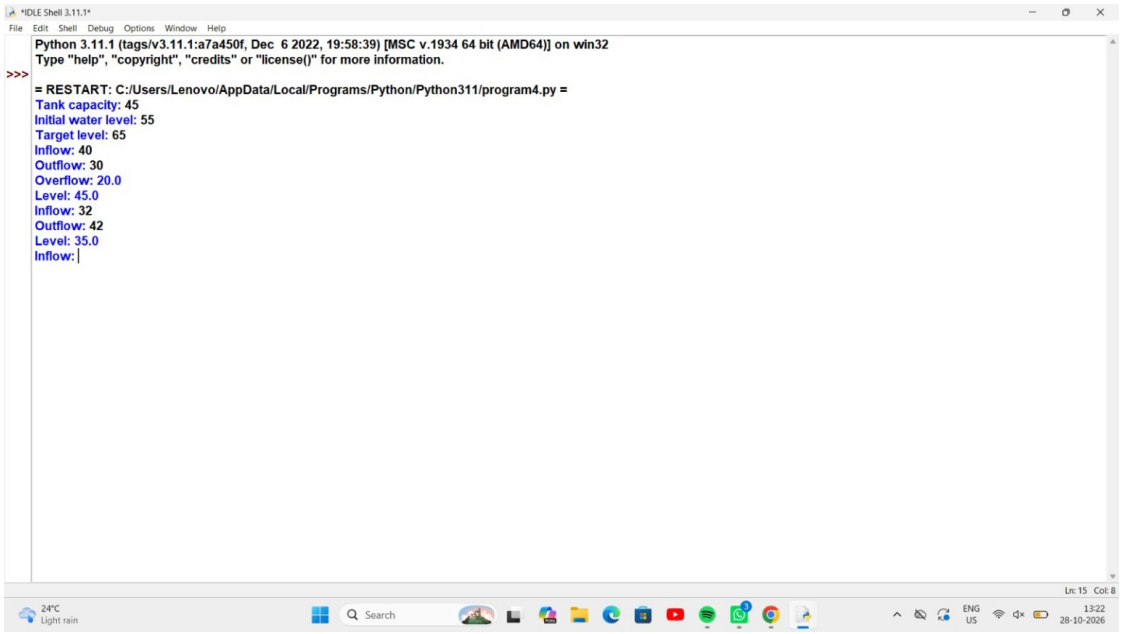
    print("Level:", level)
    if level >= target:
        print("Target reached at interval", i)
        break
else:
    print("Target not reached")
```

Sample Input and Output

Sample Input
Tank capacity: 45
Initial water level: 35
Target level: 40
Interval 1 inflow: 10
Interval 1 outflow: 2

Sample Output
Level: 43.0
Target reached at interval 1

Output Screenshot



Test Cases

Test / Input Condition Expected Result

Level 35, inflow 10, outflow 2, target 40 Target reached in interval 1

Level 40, inflow 20, outflow 0, capacity 50 Overflow reported; level capped at 50

Level 5, inflow 0, outflow 10 Shortage reported; level reset to 0

Conclusion

The simulation models water-level changes over time and reports overflow, shortage, and the first interval in which the target level is reached.

Original output screenshot supplied by the student.

Program 5: Student Performance Trend Analysis

Program Statement

Compare first and second examination scores across subjects, calculate changes and average performance, identify subjects that remain weak, and classify overall performance.

Objectives

- Collect scores for multiple subjects and two exams.
- Calculate score changes between exams.
- Flag subjects scoring below the weak-subject threshold in both exams.
- Calculate average second-exam score and classify performance.

Assumptions

- The subjects are Python, Maths, and English.
- Scores are entered as integers from 0 to 100.
- A subject is consistently weak when both exam scores are below 40.
- Classification thresholds are Excellent ≥75, Good ≥60, Average ≥40, otherwise Needs improvement.

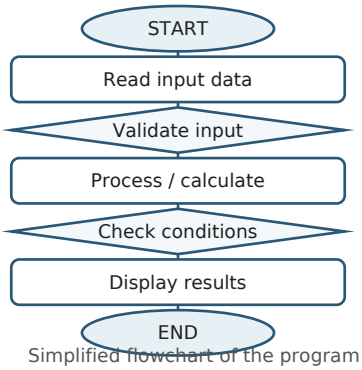
Algorithm

1. Set the subject list and total to zero.
2. For each subject, read first- and second-exam scores.
3. Add the second-exam score to the total and print the score change.
4. If both scores are below 40, report the subject as consistently weak.
5. Calculate the average second-exam score.
6. Classify and print the overall result.

Pseudo Code

```
START
subjects ← [Python, Maths, English]; total ← 0
FOR each subject
INPUT first_score, second_score
total ← total + second_score
PRINT second_score − first_score
IF first_score < 40 AND second_score < 40: PRINT weak
average ← total / number_of_subjects
CLASSIFY average by thresholds
PRINT average and classification
STOP
```

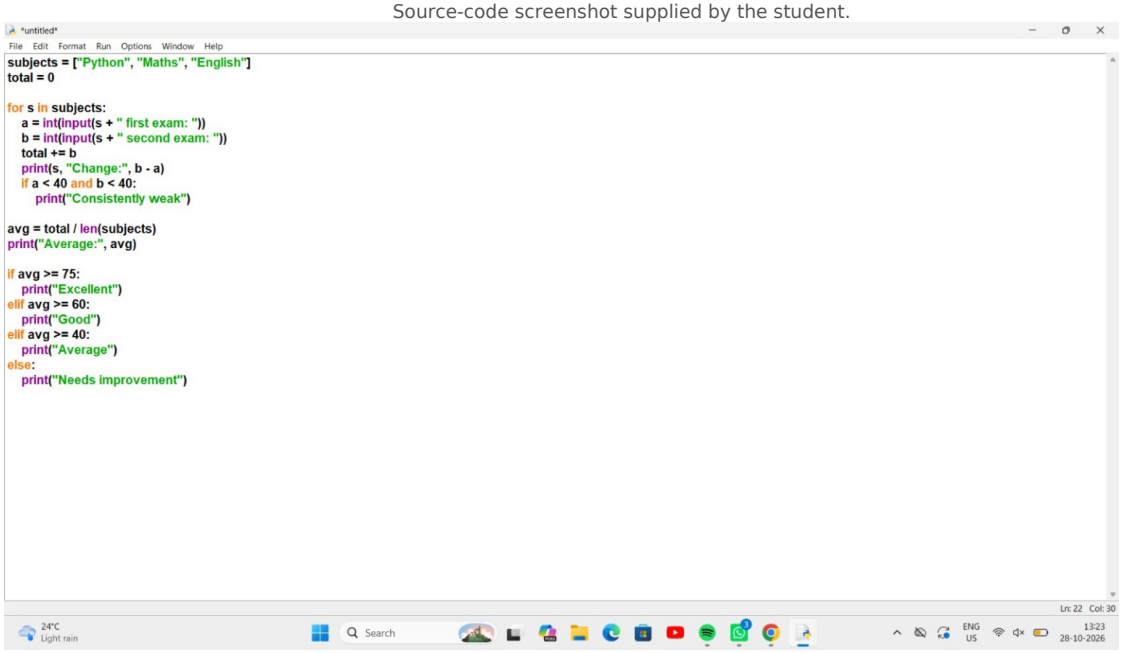
Flowchart



Simplified flowchart of the program logic.

Python Source Code (Code Screenshot)

Source-code screenshot supplied by the student.



```
subjects = ["Python", "Maths", "English"]
total = 0

for s in subjects:
    a = int(input(s + " first exam: "))
    b = int(input(s + " second exam: "))
    total += b
    print(s, "Change:", b - a)
    if a < 40 and b < 40:
        print("Consistently weak")

avg = total / len(subjects)
print("Average:", avg)

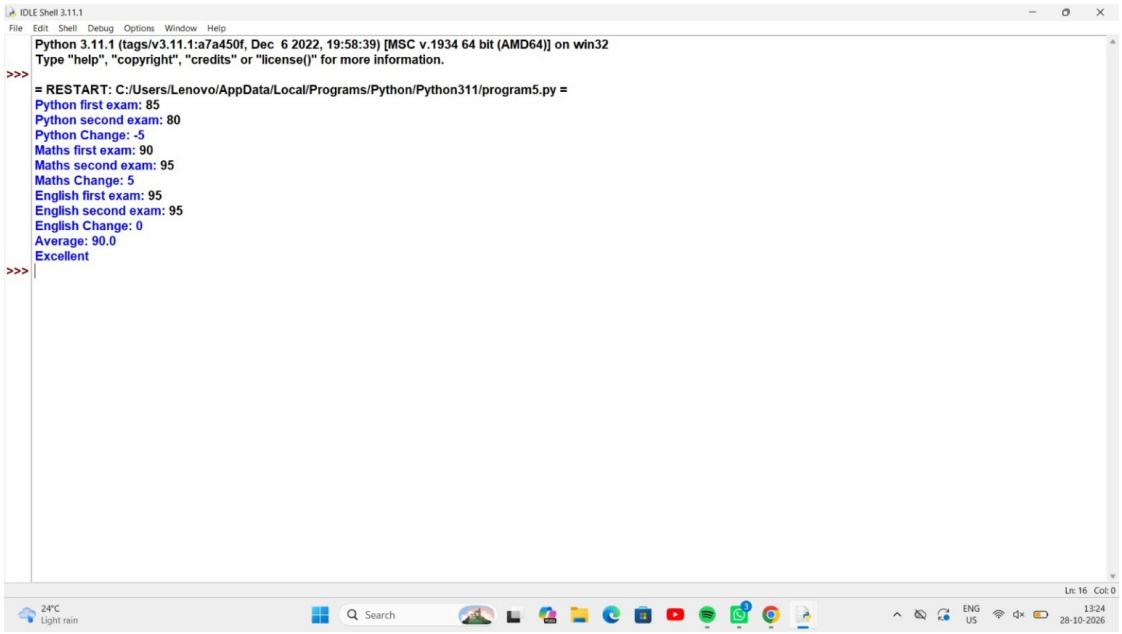
if avg >= 75:
    print("Excellent")
elif avg >= 60:
    print("Good")
elif avg >= 40:
    print("Average")
else:
    print("Needs improvement")
```

Sample Input and Output

Sample Input
Python: 85, 80
Maths: 90, 95
English: 95, 95

Sample Output
Python Change: -5
Maths Change: 5
English Change: 0
Average: 90.0
Excellent

Output Screenshot



```
Python 3.11.1 (tags/v3.11.1:a7a450f, Dec  6 2022, 19:58:39) [MSC v.1934 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: C:/Users/Lenovo/AppData/Local/Programs/Python/Python311/program5.py =
Python first exam: 85
Python second exam: 80
Python Change: -5
Maths first exam: 90
Maths second exam: 95
Maths Change: 5
English first exam: 95
English second exam: 95
English Change: 0
Average: 90.0
Excellent
>>>
```

Test Cases

Test / Input Condition Expected Result

Scores 85/80, 90/95, 95/95 Average 90; Excellent

Subject scores 20 and 35 Consistently weak warning

Second-exam scores 40, 60, 80 Average 60; Good

Conclusion

The program summarizes changes between examinations, identifies consistently weak subjects, and classifies the student's performance using the average of the latest scores.

Original output screenshot supplied by the student.

Program 6: Fuel Consumption and Route Comparison

Program Statement

Compare several vehicles for a journey by calculating required fuel and fuel cost, checking whether available fuel covers the journey plus a reserve, and selecting the lowest-cost feasible vehicle.

Objectives

- Calculate fuel required from distance and mileage.
- Estimate journey fuel cost for each vehicle.
- Check the available fuel against journey fuel plus reserve.
- Choose the feasible vehicle with the lowest calculated cost.

Assumptions

- Distance is shared by all vehicles.
- Mileage is measured in kilometres per litre.
- Fuel price is per litre.
- A vehicle is feasible only when available fuel is at least journey fuel plus reserve.

Algorithm

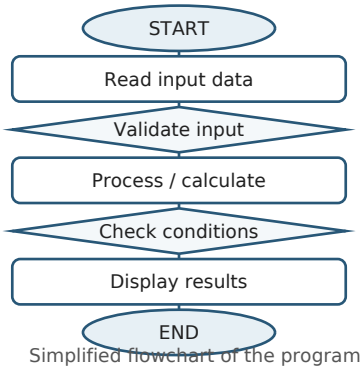
1. Read journey distance and number of vehicles.
2. For each vehicle, read name, mileage, fuel price, available fuel, and reserve fuel.
3. Reject records with non-positive mileage or negative fuel/price values.
4. Calculate fuel required and cost.
5. If available fuel covers required fuel plus reserve, consider the vehicle for the best option.
6. Print each feasible journey cost and the lowest-cost feasible vehicle.

Pseudo Code

```
START
INPUT distance, number_of_vehicles
best ← NONE
FOR each vehicle
INPUT name, mileage, price, available, reserve
IF mileage ≤ 0 OR price < 0: PRINT invalid; CONTINUE
fuel ← distance / mileage
cost ← fuel × price
IF available ≥ fuel + reserve
PRINT cost
IF best is NONE OR cost < best.cost: best ← (name, cost)
PRINT best or NONE FEASIBLE
STOP
```

— Implementation

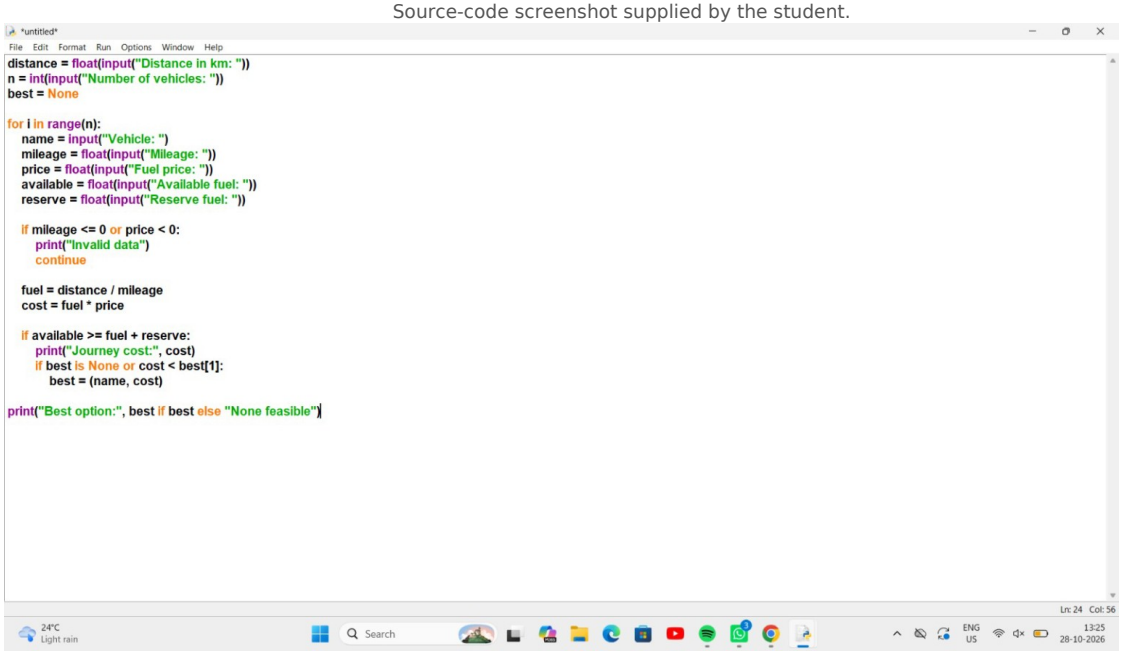
Flowchart



Simplified flowchart of the program logic.

Python Source Code (Code Screenshot)

Source-code screenshot supplied by the student.



```
distance = float(input("Distance in km: "))
n = int(input("Number of vehicles: "))
best = None

for i in range(n):
    name = input("Vehicle: ")
    mileage = float(input("Mileage: "))
    price = float(input("Fuel price: "))
    available = float(input("Available fuel: "))
    reserve = float(input("Reserve fuel: "))

    if mileage <= 0 or price < 0:
        print("Invalid data")
        continue

    fuel = distance / mileage
    cost = fuel * price

    if available >= fuel + reserve:
        print("Journey cost:", cost)
        if best is None or cost < best[1]:
            best = (name, cost)

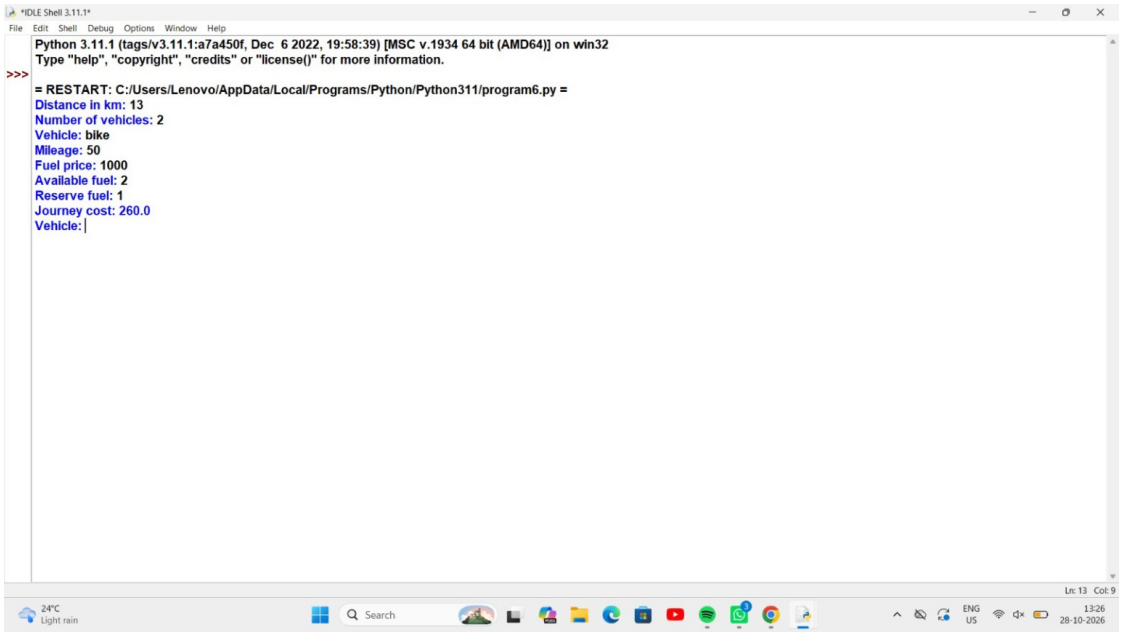
print("Best option:", best if best else "None feasible")
```

Sample Input and Output

Sample Input
Distance: 13 km
Vehicle: bike; mileage: 50; fuel price: 1000; available fuel: 2; reserve: 1

Sample Output
Journey cost: 260.0
Best option: ('bike', 260.0)

Output Screenshot



Test Cases

Test / Input Condition Expected Result

Distance 100 km; mileage 20; price 100; fuel Fuel 5 L; cost 500; feasible 10; reserve 2

Available fuel below required + reserve Vehicle excluded

Mileage 0 Invalid data

Conclusion

The program compares fuel costs while checking reserve requirements, so it recommends only a feasible option and chooses the lowest-cost one.

Original output screenshot supplied by the student.

Program 7: Multi-Product Shopping Budget Constraint

Program Statement

Evaluate a basket of products and quantities, apply a discount when the subtotal reaches a threshold, and reduce quantities of more expensive products until the final bill fits within a fixed budget.

Objectives

- Calculate the basket total from prices and quantities.
- Apply a 10% discount when the subtotal is at least ₹1,000.
- Reduce quantities to satisfy the budget constraint.
- Print the adjusted quantities and final bill.

Assumptions

- Prices are non-negative and quantities are non-negative integers.
- A 10% discount applies when the current subtotal is at least ₹1,000.
- When over budget, quantities are reduced from the highest-priced product first.
- The sample uses a budget of ₹5,000.

Algorithm

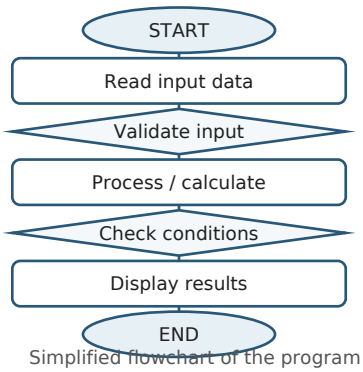
1. Read budget and number of products.
2. Read each product name, price, and quantity.
3. Calculate subtotal and apply the threshold discount.
4. Sort products by descending unit price.
5. While the discounted total exceeds the budget, reduce the current product quantity by one, moving through products in price order.
6. Print final quantities and final bill.

Pseudo Code

START
INPUT budget, n
INPUT each product name, price, quantity
FUNCTION total: subtotal ← SUM(price × quantity); apply 10% discount if subtotal ≥ 1000; RETURN
SORT products by descending price
FOR each product
WHILE total > budget AND quantity > 0: quantity ← quantity − 1
PRINT adjusted quantities and total
STOP

— Implementation

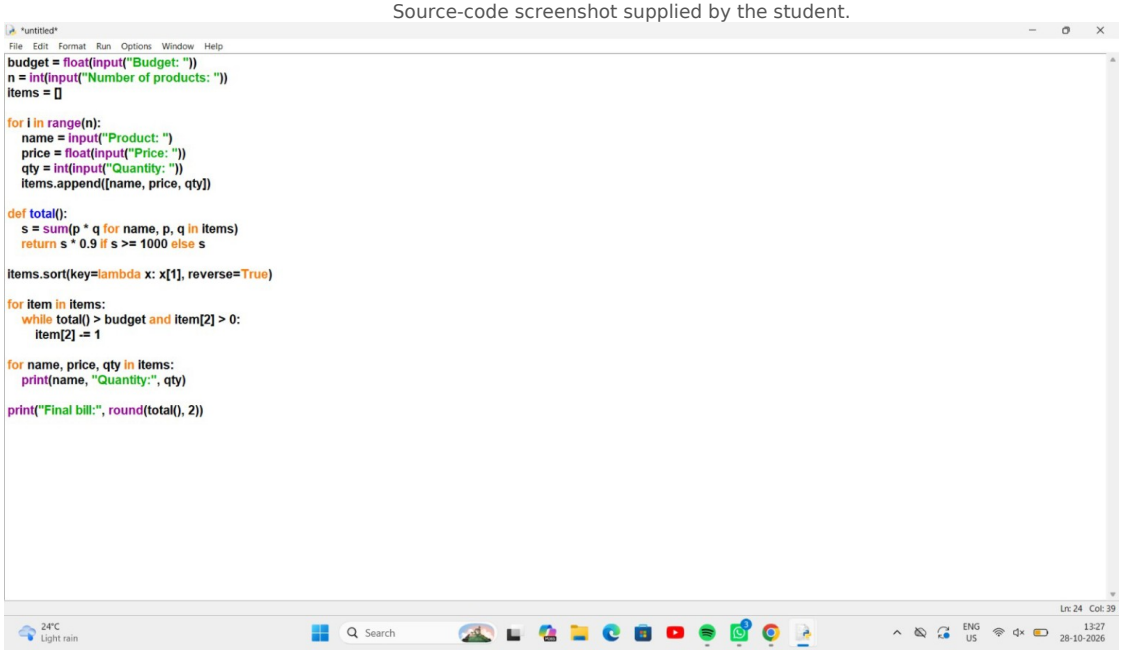
Flowchart



Simplified flowchart of the program logic.

Python Source Code (Code Screenshot)

Source-code screenshot supplied by the student.



```
budget = float(input("Budget: "))
n = int(input("Number of products: "))
items = []

for i in range(n):
    name = input("Product: ")
    price = float(input("Price: "))
    qty = int(input("Quantity: "))
    items.append([name, price, qty])

def total():
    s = sum(p * q for name, p, q in items)
    return s * 0.9 if s >= 1000 else s

items.sort(key=lambda x: x[1], reverse=True)

for item in items:
    while total() > budget and item[2] > 0:
        item[2] -= 1

for name, price, qty in items:
    print(name, "Quantity:", qty)

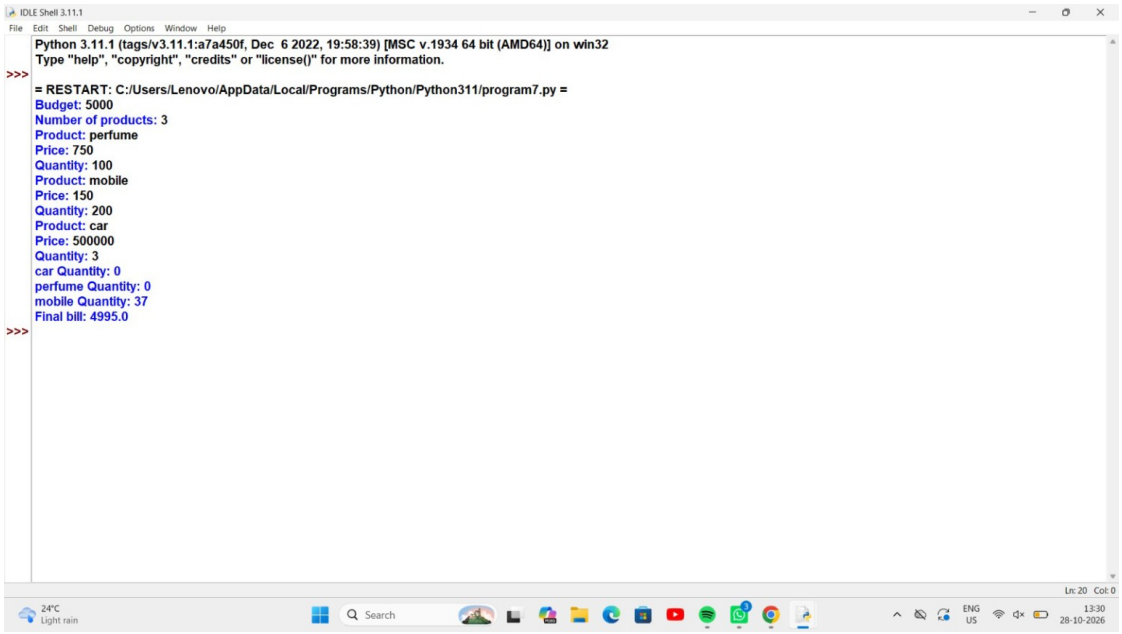
print("Final bill:", round(total(), 2))
```

Sample Input and Output

Sample Input
Budget: 5000
Perfume: price 750, quantity 100
Mobile: price 150, quantity 200
Car: price 500000, quantity 3

Sample Output
car Quantity: 0
perfume Quantity: 0
mobile Quantity: 37
Final bill: 4995.0

Output Screenshot



```
Python 3.11.1 (tags/v3.11.1:a7a450f, Dec  6 2022, 19:58:39) [MSC v.1934 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: C:/Users/Lenovo/AppData/Local/Programs/Python/Python311/program7.py =
Budget: 5000
Number of products: 3
Product: perfume
Price: 750
Quantity: 100
Product: mobile
Price: 150
Quantity: 200
Product: car
Price: 500000
Quantity: 3
car Quantity: 0
perfume Quantity: 0
mobile Quantity: 37
Final bill: 4995.0
>>>
```

Test Cases

Test / Input Condition Expected Result

Subtotal 900; budget 1000 No discount; bill 900

Subtotal 1200; budget 1000 10% discount is applied before quantity reductions

Quantity 0 Item cannot be reduced further

Conclusion

The program applies the defined discount and prioritizes reducing expensive items until the basket fits the budget, then reports the resulting quantities and bill.

Original output screenshot supplied by the student.

Program 8: University Admission Decision Engine

Program Statement

Evaluate applicants for CSE and ECE using course-specific aggregate-mark thresholds and a minimum Maths score, then rank eligible applicants by aggregate marks and Maths marks.

Objectives

- Collect applicant details and marks.
- Apply different aggregate thresholds for CSE and ECE.
- Check the minimum Maths score.
- Rank eligible candidates using aggregate marks and Maths marks as tie-breaker.

Assumptions

- CSE requires aggregate marks of at least 75; ECE requires at least 65.
- All eligible candidates must have Maths marks of at least 60.
- Marks are between 0 and 100.
- Ranking is descending by aggregate marks, then Maths marks.

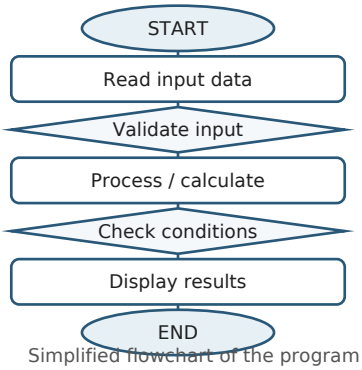
Algorithm

1. Read the number of candidates.
2. For each candidate, read name, course, aggregate marks, and Maths marks.
3. Set the minimum aggregate based on the course.
4. Add the candidate if course, aggregate, and Maths criteria are satisfied.
5. Sort eligible candidates by aggregate and Maths marks in descending order.
6. Print the ranked eligible list.

Pseudo Code

```
START
INPUT n; eligible ← empty list
FOR each candidate
INPUT name, course, aggregate, maths
minimum ← 75 if CSE else 65 if ECE
IF course valid AND aggregate ≥ minimum AND maths ≥ 60: ADD candidate
SORT eligible by aggregate then maths, descending
PRINT ranked eligible candidates
STOP
```

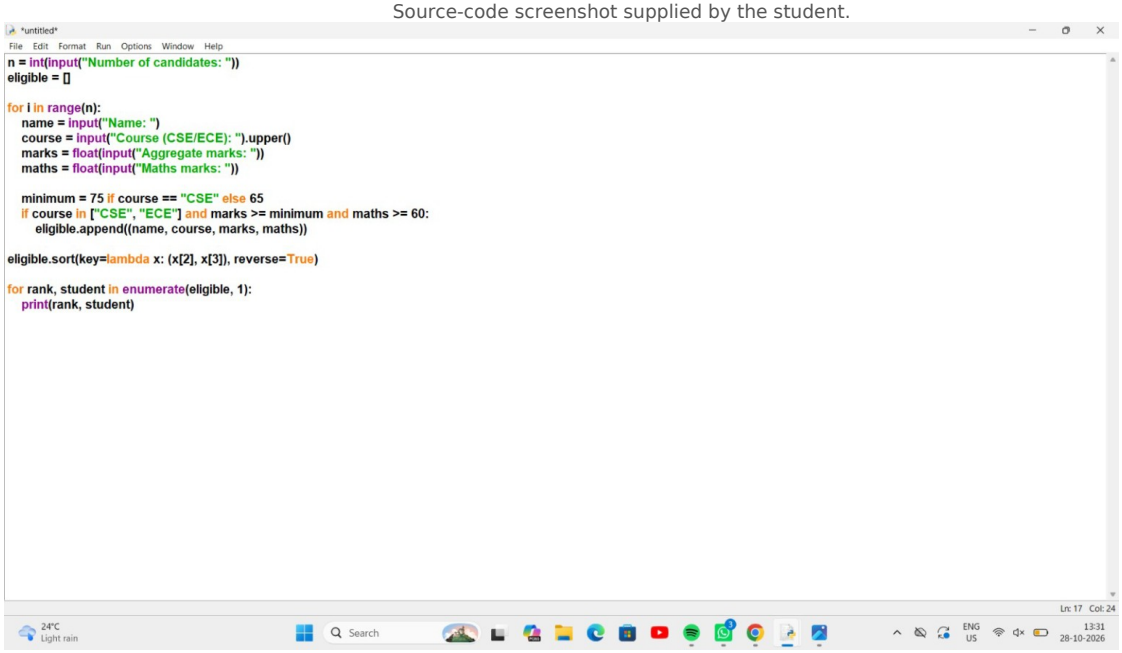
Flowchart



Simplified flowchart of the program logic.

Python Source Code (Code Screenshot)

Source-code screenshot supplied by the student.



```
n = int(input("Number of candidates: "))
eligible = []

for i in range(n):
    name = input("Name: ")
    course = input("Course (CSE/ECE): ").upper()
    marks = float(input("Aggregate marks: "))
    maths = float(input("Maths marks: "))

    minimum = 75 if course == "CSE" else 65
    if course in ["CSE", "ECE"] and marks >= minimum and maths >= 60:
        eligible.append((name, course, marks, maths))

eligible.sort(key=lambda x: (x[2], x[3]), reverse=True)

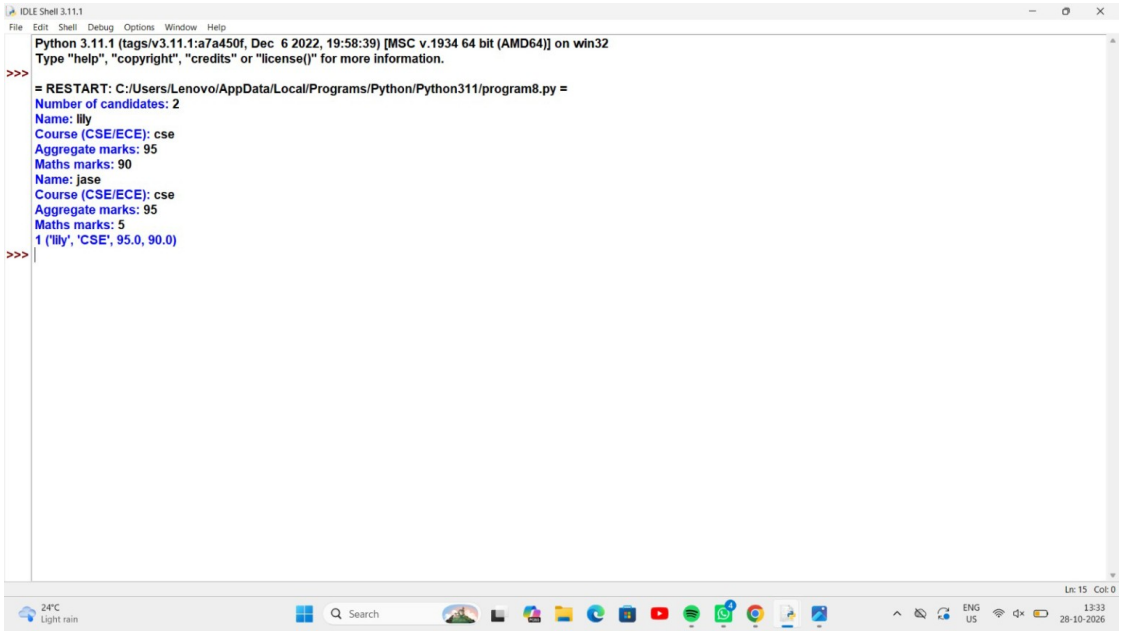
for rank, student in enumerate(eligible, 1):
    print(rank, student)
```

Sample Input and Output

Sample Input
Candidate 1: Lily, CSE, aggregate 95, Maths 90
Candidate 2: Jase, CSE, aggregate 95, Maths 5

Sample Output
1 ('Lily', 'CSE', 95.0, 90.0)

Output Screenshot



```
Python 3.11.1 (tags/v3.11.1:a7a450f, Dec  6 2022, 19:58:39) [MSC v.1934 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: C:/Users/Lenovo/AppData/Local/Programs/Python/Python311/program8.py =
Number of candidates: 2
Name: lily
Course (CSE/ECE): cse
Aggregate marks: 95
Maths marks: 90
Name: jase
Course (CSE/ECE): cse
Aggregate marks: 95
Maths marks: 5
1 ('lily', 'CSE', 95.0, 90.0)
>>>
```

Test Cases

Test / Input Condition Expected Result

CSE aggregate 80, Maths 70 Eligible

ECE aggregate 70, Maths 60 Eligible

CSE aggregate 90, Maths 50 Not eligible

Conclusion

The program filters applicants using course-specific eligibility rules and produces a ranked list using aggregate and Maths marks.

Original output screenshot supplied by the student.

Program 9: Traffic Violation Penalty Calculator

Program Statement

Process multiple traffic violations using offence type and repeat-offence status, calculate the penalty from predefined fine rules, and flag cases requiring disqualification review.

Objectives

- Define a fine table for common offences.
- Apply a doubled fine for repeat offences.
- Reject unknown offences or invalid repeat-status entries.
- Calculate the total penalty and identify disqualification review conditions.

Assumptions

- Base fines are speeding ₹1,000, signal violation ₹1,500, and no helmet ₹500.
- A repeat offence doubles the base fine.
- A review is required when three or more violations are entered.
- Vehicle category is not used in this simplified model.

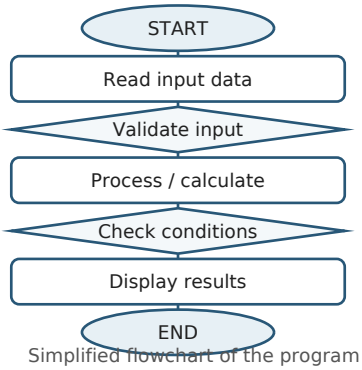
Algorithm

1. Define offence-to-fine mapping and initialize total penalty.
2. Read the number of violations.
3. For each violation, read offence and repeat status.
4. If the offence or repeat status is invalid, report invalid violation.
5. Otherwise apply the base fine and double it if repeated.
6. Add each fine to the total and print it.
7. Print total penalty and a review warning if there are at least three violations.

Pseudo Code

```
START
fines ← speeding:1000, signal:1500, no_helmet:500
INPUT count; total ← 0
FOR each violation
INPUT offence, repeat_status
IF offence unknown OR repeat invalid: PRINT invalid
ELSE fine ← fines[offence]; IF repeat=yes: fine ← 2 × fine; total ← total + fine; PRINT fine
PRINT total
IF count ≥ 3: PRINT review required
STOP
```

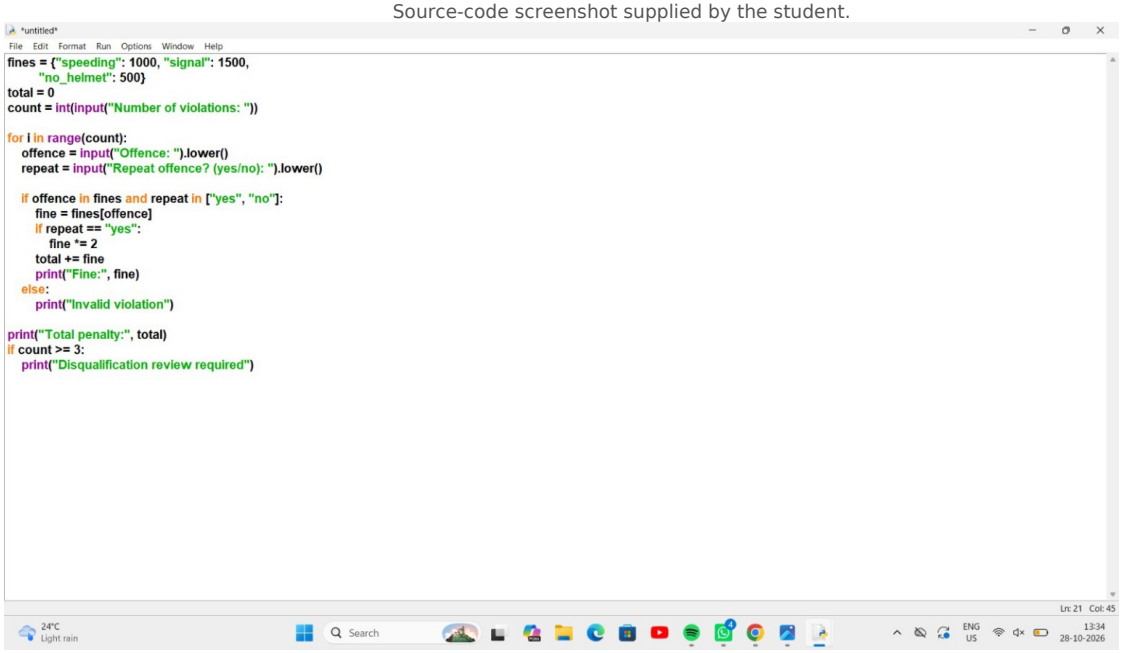
Flowchart



Simplified flowchart of the program logic.

Python Source Code (Code Screenshot)

Source-code screenshot supplied by the student.



```
fines = {"speeding": 1000, "signal": 1500,
        "no_helmet": 500}
total = 0
count = int(input("Number of violations: "))

for i in range(count):
    offence = input("Offence: ").lower()
    repeat = input("Repeat offence? (yes/no): ").lower()

    if offence in fines and repeat in ["yes", "no"]:
        fine = fines[offence]
        if repeat == "yes":
            fine *= 2
        total += fine
        print("Fine:", fine)
    else:
        print("Invalid violation")

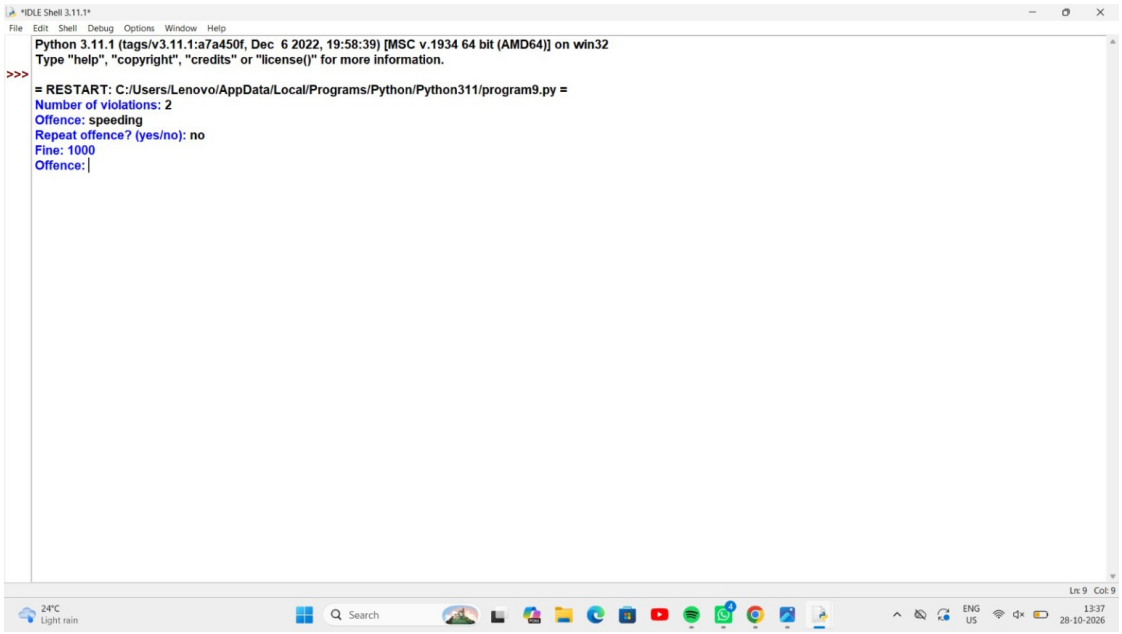
print("Total penalty:", total)
if count >= 3:
    print("Disqualification review required")
```

Sample Input and Output

Sample Input
Number of violations: 2
Offence: speeding
Repeat offence? no
Offence: signal
Repeat offence? yes

Sample Output
Fine: 1000
Fine: 3000
Total penalty: 4000

Output Screenshot



```
Python 3.11.1 (tags/v3.11.1:a7a450f, Dec  6 2022, 19:58:39) [MSC v.1934 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: C:/Users/Lenovo/AppData/Local/Programs/Python/Python311/program9.py =
Number of violations: 2
Offence: speeding
Repeat offence? (yes/no): no
Fine: 1000
Offence:
```

Test Cases

Test / Input Condition Expected Result

Speeding, not repeat Fine ₹1000

Signal, repeat Fine ₹3000

Three violations entered Review warning shown

Conclusion

The program calculates penalties consistently from a predefined fine table, accounts for repeat offences, and flags cases that need further review.

Original output screenshot supplied by the student.

Program 10: Laboratory Chemical Inventory Validation

Program Statement

Validate hypothetical chemical inventory records against allowed storage categories, maximum quantities, and compatibility restrictions. Flag quantities beyond limits and identify incompatible pairs.

Objectives

- Collect chemical names, categories, quantities, and maximum limits.
- Reject negative quantities and limits.
- Compare quantity with the stated maximum.
- Check pairs of chemicals against the provided incompatibility rules.

Assumptions

- The compatibility rules in the program are illustrative: acid/base and oxidizer/flammable are incompatible category pairs.
- Each record provides its own maximum permitted quantity.
- This program is a classroom validation example, not operational chemical-safety guidance.
- Categories are entered as acid, base, oxidizer, or flammable.

Algorithm

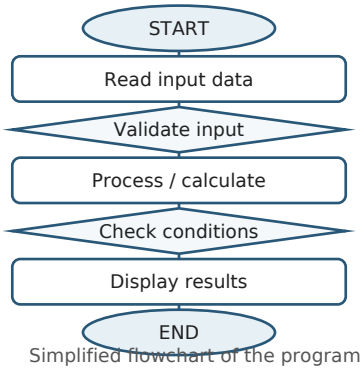
1. Define incompatible category pairs and an empty chemical list.
2. Read the number of chemicals.
3. For each chemical, read its name, category, quantity, and maximum quantity.
4. Reject negative quantity or maximum values.
5. Store valid records and report whether the quantity limit is exceeded.
6. Compare every pair of stored chemicals.
7. Print any incompatible pairs.

Pseudo Code

```
START
incompatible ← {(acid,base),(oxidizer,flammable)}
INPUT n; chemicals ← empty
FOR each chemical
INPUT name, category, quantity, maximum
IF quantity < 0 OR maximum < 0: PRINT invalid; CONTINUE
STORE record
IF quantity > maximum: PRINT limit exceeded ELSE PRINT valid
FOR each distinct pair (a,b)
IF (a.category,b.category) or reverse is incompatible: PRINT pair
STOP
```

— Implementation

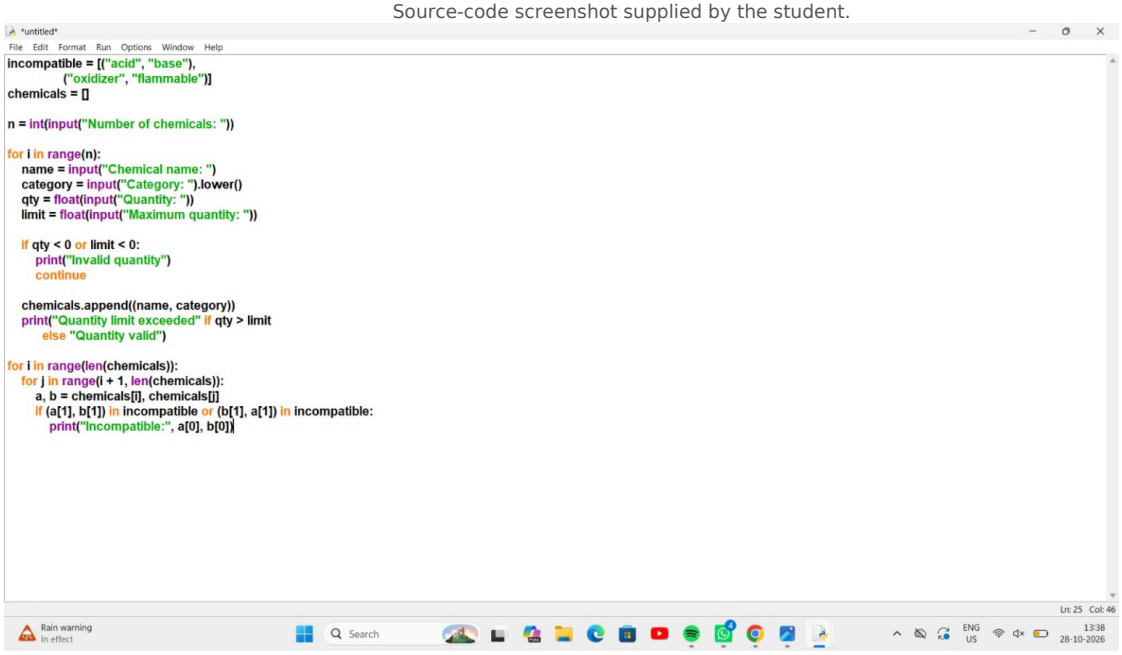
Flowchart



Simplified flowchart of the program logic.

Python Source Code (Code Screenshot)

Source-code screenshot supplied by the student.



```
incompatible = [("acid", "base"),
                ("oxidizer", "flammable")]
chemicals = []

n = int(input("Number of chemicals: "))

for i in range(n):
    name = input("Chemical name: ")
    category = input("Category: ").lower()
    qty = float(input("Quantity: "))
    limit = float(input("Maximum quantity: "))

    if qty < 0 or limit < 0:
        print("Invalid quantity")
        continue

    chemicals.append((name, category))
    print("Quantity limit exceeded" if qty > limit
          else "Quantity valid")

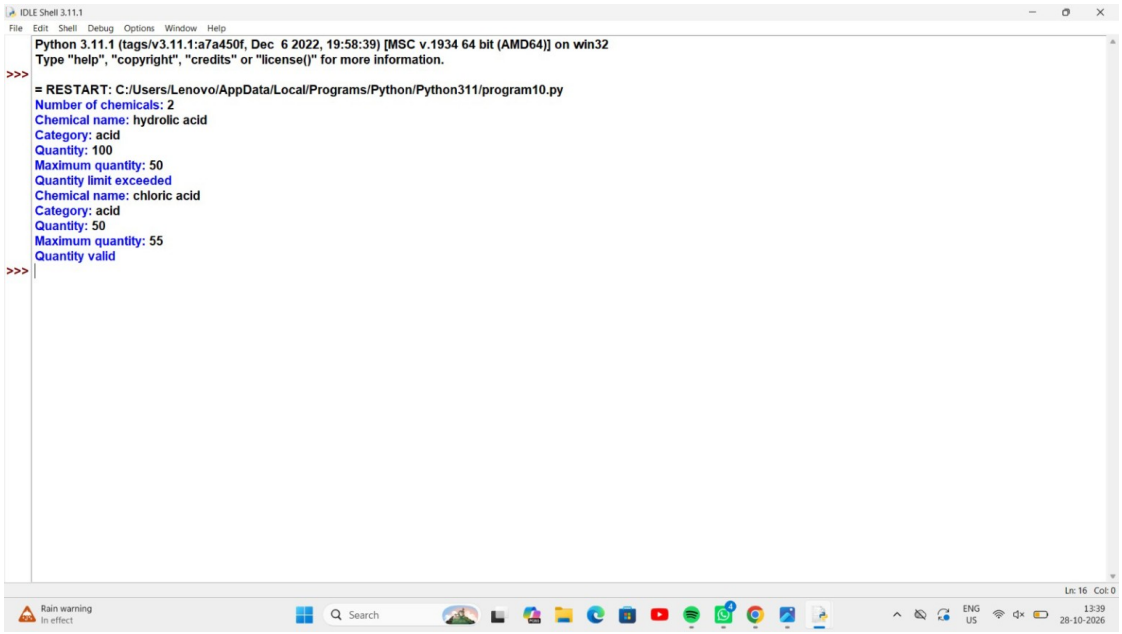
for i in range(len(chemicals)):
    for j in range(i + 1, len(chemicals)):
        a, b = chemicals[i], chemicals[j]
        if (a[1], b[1]) in incompatible or (b[1], a[1]) in incompatible:
            print("Incompatible:", a[0], b[0])
```

Sample Input and Output

Sample Input
Chemical 1: hydrochloric acid, category acid, quantity 100, maximum 50
Chemical 2: sodium hydroxide, category base, quantity 50, maximum 55

Sample Output
Quantity limit exceeded
Quantity valid
Incompatible: hydrochloric acid sodium hydroxide

Output Screenshot



Test Cases

Test / Input Condition Expected Result

Acid quantity 20, limit 50 Quantity valid

Quantity 100, limit 50 Limit exceeded

Acid and base records Incompatible pair flagged

Conclusion

The program validates inventory quantities and flags category pairs listed as incompatible in its illustrative rule set. Real laboratory handling must follow approved institutional safety procedures.

Original output screenshot supplied by the student.